CartPole, end to end

Exact workflow and hands-on guide

What you will see

You will inspect observations, compare a fixed controller with our trained actor, see actual action probabilities, watch a simulation, train a small new policy, and inspect learning curves. These activities run on the laptop. They do not test the physical Wall-E robot.

1. The system we are controlling

CartPole is a cart with a hinged pole. The cart can be pushed left or right; the controller must keep the pole upright without leaving the track. One step advances 0.02 seconds of simulated time in our installed environment.

Observation	Unit	Purpose
Cart position	m	Know distance from track center
Cart velocity	m/s	Know direction and speed
Pole angle	rad	Know tilt from upright
Pole angular velocity	rad/s	Know whether tilt is changing

Action 0 pushes left; action 1 pushes right. The force magnitude is fixed. Default reward is +1 per step, including the failure step. Failure occurs beyond approximately $\pm 12^\circ$ pole angle or ± 2.4 m cart position. The episode time limit is 500 steps.

2. Keep the roles distinct

- **Gymnasium/CartPole:** maintains simulated state and returns the consequences of an action.
- **Actor:** maps the four observations to probabilities for left and right. Training samples actions; deterministic evaluation picks the most probable.
- **Critic:** predicts discounted future return. It supports learning rather than choosing the action.
- **PPO:** collects experience and changes network weights.
- **Callbacks:** save models and periodically evaluate them.
- **TensorBoard:** displays recorded metrics; it does not train the model.

3. The two loops

Interaction: observation → actor → action → physics update → next observation and reward.

Learning: collect transitions → estimate advantages and return targets → optimize actor and critic → collect fresh transitions.

Inference runs only the action-selection loop with fixed weights. Receiving a reward does not automatically make a policy learn.

4. What training/train.py does, in order

1. Read `--config`, `--run-dir`, and `--timesteps`. Defaults come from `configs/ppo.yaml`.
2. Create a new run folder. Existing folders are rejected, preventing accidental reuse. Save effective configuration and dependency records.
3. Create separate training and validation CartPole environments, initially seeded with 42 and 1042. Monitor wrappers record episode rewards and lengths.
4. Construct PPO with `MlpPolicy`, CPU execution, seed 42, and TensorBoard logging.
5. Configure checkpoint and validation callbacks, then call `model.learn(...)`.
6. Save `final_model.zip`. Evaluate this final policy over 30 episodes with seeds 10042–10071 and write `final_evaluation.json`.
7. Close the environments, including when an error occurs inside the training block.

5. What one PPO cycle does

With our single environment, `n_steps=1024` collects 1,024 transitions with fixed policy weights. The buffer contains observations, actions, rewards, episode-boundary information, critic predictions, and old action log probabilities.

$$\delta_t = r_t + \gamma \hat{V}(s_{t+1}) - \hat{V}(s_t)$$

GAE combines prediction errors across steps to estimate advantages; true terminal transitions have no future value. PPO derives critic targets from the collected experience and estimated advantages.

Shuffle samples into minibatches of 64. Make 10 passes: $1024/64 = 16$ minibatches per pass and $16 \times 10 = 160$ optimizer steps per rollout. These optimizer steps reuse recorded data; they are not additional environment interactions. Collect a new rollout afterward.

6. Actual networks and settings

The installed default actor is $4 \to 64 \to 64 \to 2$ action scores, converted to categorical probabilities. The separate critic is $4 \to 64 \to 64 \to 1$ value. Both hidden networks use Tanh; the optimizer is Adam. We do not override the default network architecture.

Learning rate	0.0003
Discount γ	0.99
GAE λ	0.95
Actor clipping	0.2
Entropy coefficient	0.0

Clipping changes the actor's learning objective; it is not a hard clamp on probabilities or weights. Zero entropy coefficient removes the extra entropy bonus, not stochastic action sampling.

7. What happens every 10,000 training steps?

Our single-environment setup invokes both callbacks every 10,000 collection steps. This can happen partway through a rollout, rather than exactly at an optimizer boundary. **CheckpointCallback** saves a model snapshot to `checkpoints/`, such as `rl_model_10000_steps.zip`.

EvalCallback runs 10 full episodes in the separate validation environment, selecting actions deterministically. It records results in `validation/evaluations.npz`. If the mean reward is strictly better than before, it saves `best/best_model.zip`. Validation experience is not used to update weights.

Callback frequencies count callback calls. With parallel environments, a call represents multiple transitions and frequencies need adjustment. We currently use one environment.

8. Find these files in each run

config.yaml Effective settings, including command-line timestep overrides.

versions.json Python, library, and platform versions.

requirements-freeze.txt Installed dependencies for reproduction.

train_monitor.csv Training episode rewards and lengths.

validation_monitor.csv Validation episode rewards and lengths.

tensorboard/ Learning and evaluation metrics.

checkpoints/ Periodic snapshots, without performance selection.

best/best_model.zip Highest validation-mean policy seen.

final_model.zip Policy after training finishes; may differ from best.

final_evaluation.json Final policy's fresh-episode results and actual training steps.

9. Existing experiment and its limits

The saved run is `experiments/cartpole-ppo-seed42-100k/`. Its 100,000-step request completed 100,352 steps because PPO finished the last 1,024-step rollout. The recorded final and separate holdout evaluations each scored 500.0 ± 0.0 over 30 episodes.

That establishes a CartPole workflow milestone. It does not establish physical balance. In particular, the real robot lacks encoders, so its observations cannot automatically include CartPole-like wheel position and velocity.

10. Evaluation score versus critic value

The episode score sums rewards without discounting. The critic estimates discounted return: 500 certain remaining rewards of 1 at $\gamma = 0.99$ sum to about 99.34, not 500. A critic prediction is approximate and depends on the policy.

11. Start in VS Code's terminal

Run these commands from the project root:

```
cd /home/varun/Intelligent_systems/Wall-E/wall-e-rl
source .venv/bin/activate
```

The environment already contains the training dependencies. All commands below use its Python. Commands that load a saved model do not retrain it.

12. First see what a fixed controller does

```
python -m training.demo --controller right
```

At seed 20042, verified locally: 10 steps, return 10, true termination. Look at the increasing rightward cart velocity and growing leftward pole rotation. Always pushing right ignores the changing state.

13. Inspect the trained actor

```
python -m training.demo --controller ppo --print-every 1
```

This prints every step. For less output, omit `--print-every 1`; the default prints the first five, every fiftieth, and the ending step.

Locally verified first two decisions:

```
step 1: P(left)=0.993 P(right)=0.007 -> LEFT
step 2: P(left)=0.366 P(right)=0.634 -> RIGHT
```

The printed probabilities apply to the observation *before* the action. The numbers after `next=` describe the state *after* that action. They are in order: position, velocity, angle, angular velocity. Rounding can hide small differences. At seed 20042 the deterministic policy reaches 500 steps with `truncated=True`, equivalent to 10 seconds of simulated time.

14. Open the animation

The rendering dependency `pygame` was absent when this guide was built. Install the extra once, then watch from a desktop terminal:

```
python -m pip install 'gymnasium[classic-control]'
python -m training.demo --render
python -m training.demo --controller right --render
```

The fixed controller fails quickly; the PPO attempt lasts longer. GUI rendering was not tested during guide creation. If a display is unavailable, omit `--render`; the numerical example still works. Watch how motor-like pushes change motion over several steps, not only the pole angle immediately.

15. Try another starting state

```
python -m training.demo --seed 20043
python -m training.demo --controller right --seed 20043
```

The seed changes the initial randomized state. Compare both controllers using the same seed so that the starting situation is matched. Different versions or execution conditions can affect reproducibility.

16. Try stochastic action selection

```
python -m training.demo --stochastic --print-every 1
```

This samples from the learned distribution instead of picking the more probable action. The weights remain fixed. A less probable action can occur; that is not a bug. The demo seeds action-sampling randomness too, so the same seed is intended to repeat the same experiment on a compatible setup. To explore another sequence, change the seed.

17. Evaluate several episodes without animation

```
python -m training.evaluate \
  experiments/cartpole-ppo-seed42-100k/final_model.zip \
  --episodes 30 --seed 30042
```

This prints mean reward, spread, and episode-length range. These are new evaluation seeds; this guide does not promise a particular result. Evaluation never changes weights.

18. Run a short new training experiment

```
python -m training.train --timesteps 2048
```

This builds fresh networks and performs two rollout/update cycles. It saves a new timestamped run and evaluates its final policy. Expect a short run to be weaker or variable; 2,048 steps check the workflow, not completion of learning. With the default 10,000-step callback frequencies, this short run has no periodic checkpoint or callback evaluation. It still saves a final model and runs final evaluation.

The program prints the run path. Replace `<new-run>` below with that actual folder name; do not type the angle brackets literally:

```
python -m training.demo \
  --model experiments/<new-run>/final_model.zip --render
```

Compare the short-run actor with the existing 100k actor from the same seed. More training does not guarantee improvement on every individual episode.

19. Train long enough to see the callbacks

```
python -m training.train --timesteps 20000
```

This starts another fresh run, not a continuation of the earlier short run. With 1,024-step blocks it completes 20,480 steps. Checkpoint and validation events occur at 10,000 and 20,000 collection steps. Actual runtime depends on the machine; a specific final score is not guaranteed.

20. Open TensorBoard in a second terminal

```
cd /home/varun/Intelligent_systems/Wall-E/wall-e-rl
source .venv/bin/activate
tensorboard --logdir experiments --port 6006
```

Open `http://localhost:6006` in your browser. Select the run you want to inspect. Stop the server with Ctrl+C when finished.

rollout/ep_rew_mean Recent training episode rewards. Higher usually means longer CartPole survival; it is not an independent evaluation.

rollout/ep_len_mean Recent episode lengths. In this reward scheme they closely track episode returns.

eval/mean_reward Periodic deterministic validation performance. A 2,048-step run has no such callback points with our default frequencies.

train/value_loss How closely critic predictions fit their training targets. Lower alone does not establish a better policy.

train/approx_kl A measure of policy-distribution change during optimization.

train/clip_fraction Fraction of sampled probability ratios outside the configured clipping interval.

train/entropy_loss Related to action-distribution uncertainty; it is negative entropy in this implementation, not episode reward.

TensorBoard smoothing can hide variation. Inspect unsmoothed values and the separate evaluation results, not just a smooth rising curve.

21. A useful completion checklist

Can you identify the four observation entries? Distinguish pre-action probabilities from the next observation? Explain a positive angle with negative angular velocity? Tell failure from time-limit truncation? Locate the saved model and evaluation JSON? Explain why inference leaves the weights unchanged?

Files and references

`training/demo.py`: runnable numerical and visual demonstration.

`training/train.py`: training orchestration and callbacks.

`training/evaluate.py`: fresh deterministic episodes.

`docs/CARTPOLE_GYMNASIUM.md`: detailed observation trace.

Official background: Gymnasium CartPole, Stable-Baselines3 PPO, and Spinning Up concepts. This guide's workflow and commands were checked against our local source and installed implementation.